

3/12/18

UNIT-2 "Divide & Conquer"

- General Method (or) Abstraction Method of Divide and Conquer
- Defective Chess board
- Binary search
- Quick Sort
- Merge Sort
- Finding Maximum and Minimum
- performance Measurement
- Randomized Sorting Algorithm. eg:- quick sort

Divide and Conquer (General Method):-

- * If the given problem is a small problem then return a solution
- * If the given problem is a large then apply divide and conquer method.
- * If the given problem is large then it can be divided into possible number of sub problems.
- * For each sub problem we have to find out solutions
- * Finally all the solutions are combined

Explain Divide & Conquer Method Algorithm:-

DAC(p)

{

if (small(p))

{

S(p);

}

else

{

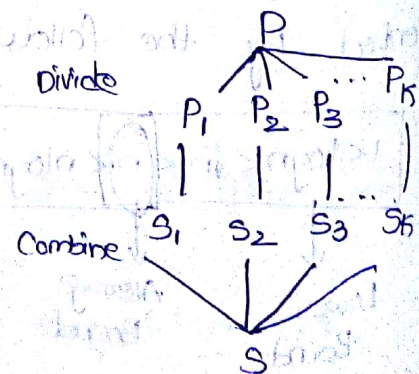
divide p into $P_1, P_2, P_3, \dots, P_K$

Apply $DA(P_1), DA(P_2), DA(P_3), \dots, DA(P_K)$

combine $DA(P_1), DA(P_2), DA(P_3), \dots, DA(P_K)$

}

Tree Representation



Note: 2

* All the divide and conquer algorithms are recursive hence we can find out the time complexity using Recurrence Relation

Binary Search:-

* In the binary search method the elements are sorted order that is ascending order.

* If we want search for the element say 'x'.

* Then we first divided the list at middle so that two sublists are created.

* If x greater than middle then right sub list consider and x is searched in the right sub list otherwise x is searched in the left sub list.

eg:- the given list of elements are 3, 6, 8, 12, 14, 17, 25, 29, 31, 36, 42, 47, 53, 55, 62. and key element is 42. Then find a key element 42 is available or not and also find time complexity

* Consider the given list of elements are

3	6	8	12	14	17	25	29	31	36	42	47	53	55	62
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

let us assume $l=1$

high $h=n=15$

then find mid value $\text{mid}(m) = \lfloor (l+h)/2 \rfloor$

if always low less than or equal to high then find mid
 $\text{mid}(m) = \frac{(l+h)}{2}$

if $(\text{key} == \text{mid})$ then $42 == 29$ hence it is false.

if $(\text{key} < \text{mid})$

$42 < 29$ false

if $(\text{key} > \text{mid})$

$42 > 29$ True

then $low = mid + 1$

$$= 8 + 1$$

$$l = 9$$

$$\text{now } l = 9, h = 15, m = \frac{9+15}{2} = \frac{24}{2} = 12.$$

if (key == mid) then

$$42 == 47 \text{ false}$$

if (key < mid) then

$$42 < 47 \text{ True.}$$

$$high = mid - 1$$

$$h = 12 - 1$$

$$h = 11$$

$$\text{now } h = 11, l = 9, m = \frac{11+9}{2} = \frac{20}{2} = 10$$

if (key == mid)

$$42 == 36 \text{ false}$$

if (key < mid) then

$$42 < 36 \text{ false}$$

if (key > mid) then

$$42 > 36 \text{ True}$$

$$low = mid + 1$$

$$= 10 + 1$$

$$l = 11$$

$$\text{now } l = 11, h = 11, m = \frac{11+11}{2} = 11$$

if (key == mid) then

$$42 == 42 \text{ True}$$

the element is found.

Now key is 42.

while (l ≤ h)

key == mid

return m

key < mid

h = mid - 1;

key > mid

l = mid + 1;

else

return 0;

The time complexity of binary search is —

	Best case	Average case	Worst case
Successful	$O(1)$	$O(\log n)$	$O(\log n)$
Unsuccessful	$O(\log n)$	$O(\log n)$	$O(\log n)$

Recursive Algorithm for Binary Search:— (Iterative)

```

int BinarySearch(A, n, key)
{
    l = 1, h = n
    while (l ≤ h)
    {
        mid = (l + h) / 2;
        if (key == A[mid])
            return mid;
        if (key < A[mid])
            h = mid - 1;
        else
            l = mid + 1;
    }
    return 0;
}

```

Recursive algorithm:

Algorithm RecursiveBinarySearch(l, h, key)

```

{
    if (l == h)
    {
        if (A[l] == key)
            return l;
        else
            return 0;
    }
    else
    {
        mid = (l + h) / 2;
        if (key == A[mid])
            return mid;
        if (key < A[mid])
            return RecursiveBinarySearch(l, mid - 1, key);
        else
            return RecursiveBinarySearch(mid + 1, h, key);
    }
}

```


Recurrence Relation:-

$$T(n) = \begin{cases} 1 & \text{if } n=1 \\ T(n/2)+1 & \text{if } n>1 \end{cases} \quad \text{prove that the time complexity for}$$

average and worst case is $O(\log n)$ and unsuccessful search in these cases is also $O(\log n)$.

consider $T(n) = T(n/2) + 1 \rightarrow \text{①}$

Now substitute n is replaced by $n/2$ in equ ① then we get

$$T(n) = (T(n/2^2) + 1) + 1$$

$$= T(n/2^2) + 2 \rightarrow \text{②}$$

$$= T(n/2^3) + 3 \rightarrow \text{③}$$

$$= T(n/2^4) + 4 \rightarrow \text{④}$$

assume $\frac{n}{2^k} = 1$ then $n = 2^k$ then $k = \log_2 n$

$$T(n) = T(n/2^k) + k$$

$$= T(1) + \log n$$

$$= 1 + \log n$$

Hence.

Time complexity is $O(\log n)$.

ex:- 11, 22, 30, 33, 40, 44, 55, 60, 66, 77, 80, 88, 99, $K=40$

1						m						l
11	22	30	33	40	44	55	60	66	77	80	88	99
1	2	3	4	5	6	7	8	9	10	11	12	13

① let us assume $l=1$ Key $K=40$

high $h=n=13$

then find mid value $\text{mid}(m) = \left\lfloor \frac{l+h}{2} \right\rfloor$

if always low less than or equal to high then find mid

$$\text{mid}(m) = \frac{l+h}{2}$$

if $(\text{key} == \text{mid})$ then $40 == 55$ hence it is false

if (key < mid)

40 < 55 true

h = mid - 1;

= 7 - 1;

h = 6

now h = 6, l = 1, m = $\frac{6+1}{2} = \frac{7}{2} = 3$

if (key == mid) then

40 == 30 false

if (key < mid) then

40 < 30 false

if (key > mid) then

40 > 30 true

l = mid + 1

= 3 + 1

= 4

now l = 4, h = 6, m = $\frac{4+6}{2} = \frac{10}{2} = 5$

if (key == mid) then

40 == 40 true

the element is found.

② now key k = 62

l = 1, h = 13, k = 62

consider if (l ≤ h)

mid = $\frac{l+h}{2} = \frac{1+13}{2} = \frac{14}{2} = 7$

if (key == mid) then

62 == 55 false

if (key < mid) then

62 < 55 false

if (key > mid)

62 > 55 true

l = 7 + 1 = 8

now l = 8, h = 13, m = $\frac{8+13}{2} = \frac{21}{2} = 10$

if (l ≤ h)

mid = $(\frac{8+13}{2}) = 10$

if (key == mid)
62 == 77 false

if (key < mid)
62 < 77 True

high = mid - 1

= 10 - 1

h = 9

Finding Maximum and Minimum element

* Identify low index and High index.

* calculate Mid value based on low and high indexes
that is mid $m = \left\lfloor \frac{l+h}{2} \right\rfloor$

* Based on the mid element the given list can be divided into two sub lists.

1) Left sub list

2) Right sub list

* consider left sub list, identify low, high indexes and calculate mid element.

* until the left sub list contain one (or) two elements in the list.

* Consider Right sub list, identify low, high indexes and calculate mid element. until you get the list contain 1 (or) 2 elements you get elements.

* compare left & right sub lists identify maximum element and minimum element.

ex:- Identify Maximum and minimum element from the following given list 50, 40, -5, -9, 45, 90, 65, 25, 75.

50	40	-5	-9	45	90	65	25	75
0	1	2	3	4	5	6	7	8

$$l=0 \quad h=8$$

$$mid = \left\lfloor \frac{0+8}{2} \right\rfloor = 4$$

50	40	-5	-9	45
0	1	2	3	4

left sub list

$$l=0, h=4$$

$$mid = \left\lfloor \frac{0+4}{2} \right\rfloor = 2$$

50	40	-5
0	1	2

$$l=0, h=2$$

$$m = \left\lfloor \frac{0+2}{2} \right\rfloor = 1$$

50	40
0	1

$$\begin{aligned} \max &= 50 \\ \min &= 40 \end{aligned}$$

-5
2

$$\begin{aligned} \max &= -5 \\ \min &= -5 \end{aligned}$$

$$\begin{aligned} \max &= 50 \\ \min &= -5 \end{aligned}$$

$$\begin{aligned} \max &= 50 \\ \min &= -9 \end{aligned}$$

$$\begin{aligned} \max &= 90 \\ \min &= -9 \end{aligned}$$

90	65	25	75
5	6	7	8

Right sub list

$$l=5, h=8$$

$$m = \left\lfloor \frac{5+8}{2} \right\rfloor = 6$$

90	65
5	6

$$\begin{aligned} \max &= 90 \\ \min &= 65 \end{aligned}$$

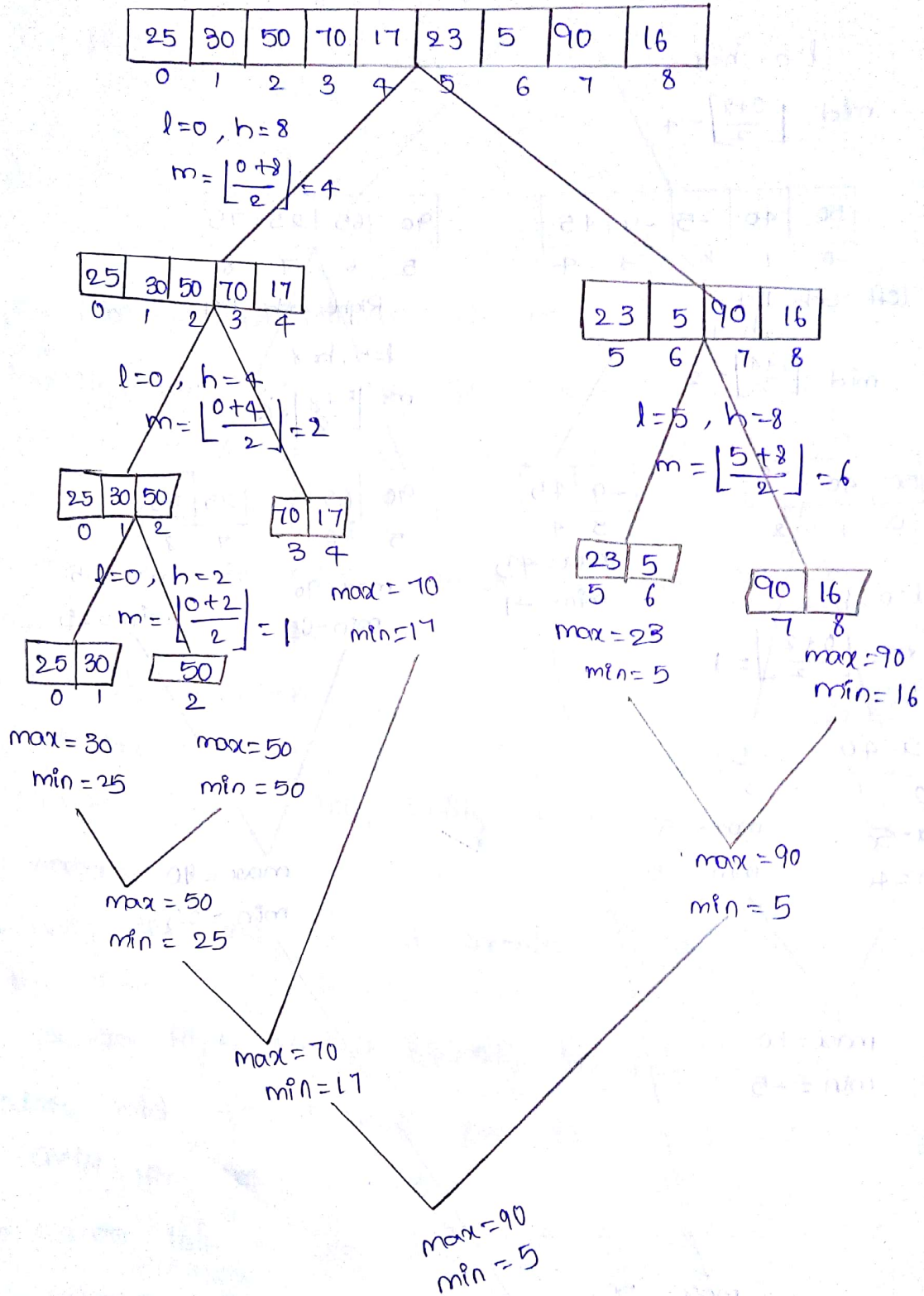
25	75
7	8

$$\begin{aligned} \max &= 75 \\ \min &= 25 \end{aligned}$$

$$\begin{aligned} \max &= 90 \\ \min &= 25 \end{aligned}$$

hence the maximum number is 90
minimum number is -9

Ex: 2 25, 30, 50, 70, 17, 23, 5, 90, 16.



Algorithm MaxMin($i, j, \max, \min$)

```
{
  if ( $i = j$ ) then
     $\max := \min := a[i]$ ; // list contain only one element
  elseif ( $i = j - 1$ )
    {
      if ( $a[i] < a[j]$ )
        {
           $\max := a[j]$ ;
           $\min := a[i]$ ;
        }
      else
        {
           $\max := a[i]$ ; // list contains 2 elements
           $\min := a[j]$ ;
        }
    }
  else
    {
       $\text{mid} = \lfloor \frac{i+j}{2} \rfloor$ ;
       $\max1 = \text{MaxMin}(i, \text{mid}, \max, \min)$ ;
       $\min1 = \text{MaxMin}(\text{mid}+1, j, \max1, \min1)$ ;
      if ( $\max < \max1$ ) then // more than one element
         $\max := \max1$ ;
      if ( $\min > \min1$ ) then
         $\min := \min1$ ;
    }
}
```

Note(3)
* The time complexity for finding maximum and minimum number is $O(n)$

* The bestcase, average case and worst case time complexity for max & min is $O(n)$.

Quick Sort:-

* In quick sort method we are going to identify i (low), j (high) and pivot.

* Always $\text{pivot} = a[i]$; a is array list of elements

* i has to be incremented when $a[i] \leq \text{pivot}$.

* j has to be decremented when $a[j] \geq \text{pivot}$.

* Otherwise no change to be made if $i \& j$ has stop then swap $a[i] \& a[j]$.

* Then again continue the process.

* If i & j crosses each other then swap $a[i]$, pivot.

* Finally, we can generate the list in sorted order i.e. ascending order.

eg: 50, 30, 10, 90, 80, 20, 40, 70.

i, j will be in same place
that situation we can swap
 j , pivot.

⇒ $\begin{matrix} i \\ 50 \\ p \end{matrix}$ 30 10 90 80 20 40 $\begin{matrix} j \\ 70 \end{matrix}$

$\Rightarrow$ 50 30 10 90 80 20 40 70
P i

$\Rightarrow 50 \quad 30 \quad 10 \quad 90 \quad 80 \quad 20 \quad 40 \quad 70$
P $\uparrow$ $\downarrow$

$\Rightarrow$ 50 30 10 90 80 20 40 70
 P
 i swap j

⇒ 50 30 10 40 80 20 90 70

⇒ 50 30 10 40 80 20 90 70

$\Rightarrow$ 50 30 10 40 20 80 90 70
 P
 ← swap

$\Rightarrow$

20	30	10	40	50	80	90	70
└──────────┘					└────────┘		
sublist ₁ < pivot					sublist ₂ > pivot		

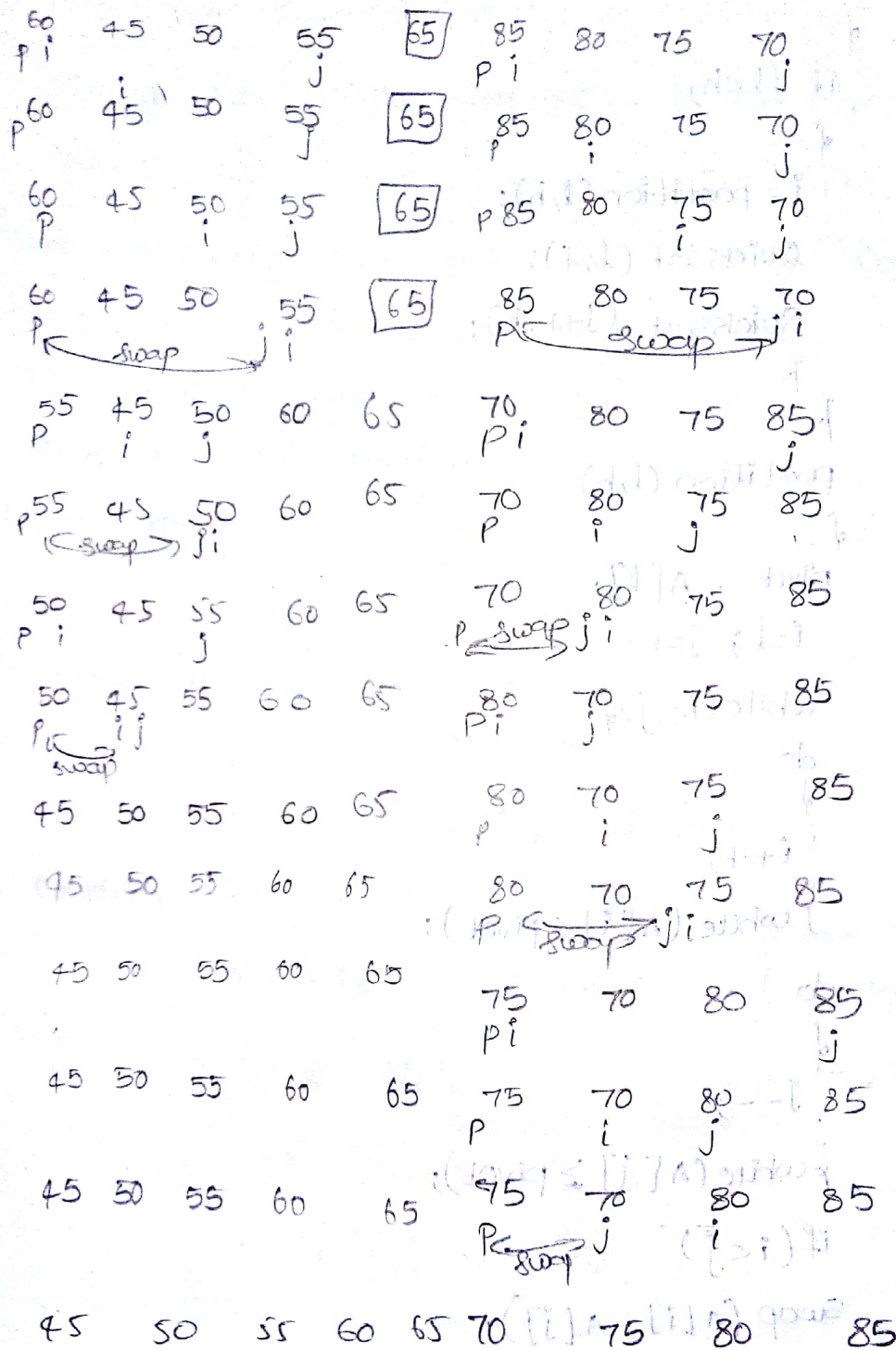
⇒ 20 30 40 50 60 70
pi j

⇒ 20 30 10 40 50 80 90 70

$\Rightarrow$ 20 10 30 40 50 80 90 70
 $\quad$ $\begin{matrix} p \\ i \end{matrix}$ $\begin{matrix} j \\ i \end{matrix}$
 $\Rightarrow$ 20 10 30 40 50 80 90 70
 $\quad$ $\begin{matrix} p \\ i \end{matrix}$ $\begin{matrix} j \\ i \end{matrix}$ $\xrightarrow{\text{swap}}$
 $\Rightarrow$ 10 20 30 40 50 $\begin{matrix} 80 \\ p \\ i \end{matrix}$ 90 70 $\begin{matrix} j \\ i \end{matrix}$
 $\Rightarrow$ 10 20 30 40 50 80 $\begin{matrix} 90 \\ i \end{matrix}$ 70 $\begin{matrix} j \\ i \end{matrix}$ $\xrightarrow{\text{swap}}$
 $\Rightarrow$ 10 20 30 40 50 80 70 $\begin{matrix} 90 \\ i \end{matrix}$ $\begin{matrix} j \\ i \end{matrix}$
 $\Rightarrow$ 10 20 30 40 50 80 70 90 $\begin{matrix} i \\ i \end{matrix}$
 $\Rightarrow$ 10 20 30 40 50 80 70 90 $\begin{matrix} i \\ i \end{matrix}$
 $\Rightarrow$ 10 20 30 40 50 80 70 90 $\begin{matrix} i \\ i \end{matrix}$ $\begin{matrix} j \\ i \end{matrix}$ $\xrightarrow{\text{swap}}$
 $\Rightarrow$ 10 20 30 40 50 70 80 90

ex: 62, 71, 72, 80, 82, 60, 52, 51, 42.

$\Rightarrow$ 62 71 72 80 82 60 52 51 42
 $\quad$ $\begin{matrix} p \\ i \end{matrix}$ $\begin{matrix} j \end{matrix}$
 $\Rightarrow$ 62 71 72 80 82 60 52 51 42
 $\quad$ $\begin{matrix} p \\ i \end{matrix}$ $\begin{matrix} j \end{matrix}$ $\xrightarrow{\text{swap}}$
 $\Rightarrow$ 62 42 72 80 82 60 52 51 71
 $\quad$ $\begin{matrix} p \\ i \end{matrix}$ $\begin{matrix} j \end{matrix}$ $\xrightarrow{\text{swap}}$
 $\Rightarrow$ 62 42 51 80 82 60 52 72 71
 $\quad$ $\begin{matrix} p \\ i \end{matrix}$ $\begin{matrix} j \end{matrix}$ $\xrightarrow{\text{swap}}$
 $\Rightarrow$ 62 42 51 82 80 60 52 72 71
 $\quad$ $\begin{matrix} p \\ i \end{matrix}$ $\begin{matrix} j \end{matrix}$ $\xrightarrow{\text{swap}}$
 $\Rightarrow$ 62 42 51 52 60 82 80 72 71
 $\quad$ $\begin{matrix} p \\ i \end{matrix}$ $\begin{matrix} j \end{matrix}$ $\xrightarrow{\text{swap}}$
 $\Rightarrow$ 60 42 51 52 62 82 80 72 71
 $\quad$ $\begin{matrix} p \\ i \end{matrix}$ $\begin{matrix} j \end{matrix}$ $\xrightarrow{\text{swap}}$
 $\Rightarrow$ 60 42 51 52 62 82 80 72 71
 $\quad$ $\begin{matrix} p \\ i \end{matrix}$ $\begin{matrix} j \end{matrix}$ $\xrightarrow{\text{swap}}$
 $\Rightarrow$ 60 42 51 52 62 82 80 72 71
 $\quad$ $\begin{matrix} p \\ i \end{matrix}$ $\begin{matrix} j \end{matrix}$ $\xrightarrow{\text{swap}}$
 $\Rightarrow$ 52 42 51 60 62 82 80 72 71
 $\quad$ $\begin{matrix} p \\ i \end{matrix}$ $\begin{matrix} j \end{matrix}$ $\xrightarrow{\text{swap}}$



10/12/18
Quick Sort Algorithm:

quickSort(l, h)

```
{
  if (l < h)
  {
    j = partition(l, h);
    quickSort(l, j);
    quickSort(j+1, h);
  }
}
```

partition(l, h)

```
{
  pivot = A[l];
  i = l; j = h;
  while (i < j)
  do
  {
    i++;
  } while (A[i] ≤ pivot);
  do
  {
    j--;
  } while (A[j] ≥ pivot);
  if (i < j)
    swap(A[i], A[j]);
  }
  swap(A[l], A[j]);
  return j;
}
```

* Time complexity for Quick Sort

Note:

Best Case Time complexity is $O(n \log n)$

Average Case Time complexity is $O(n \log n)$

Worst Case Time complexity is $O(n^2)$

* Time complexity for Best and Average Case:

To construct left sublist i.e. $T(n/2)$

To construct Right sublist i.e. $T(n/2)$

Time to build partition i.e. n

$$\begin{aligned}\text{Hence total time } T(n) &= T(n/2) + T(n/2) + n \\ &= 2T(n/2) + n\end{aligned}$$

Now the recurrence relation is

$$T(n) = \begin{cases} 1 & \text{if } n=1 \\ 2T(n/2) + n & \text{if } n>1 \end{cases}$$

$$\text{Consider } T(n) = 2T(n/2) + n \quad \text{--- (1)}$$

$$T(n) = 2 \left[2T\left(\frac{n}{2^2}\right) + \frac{n}{2} \right] + n$$

$$= 2^2 T\left(\frac{n}{2^2}\right) + n + n$$

$$= 2^2 T\left(\frac{n}{2^2}\right) + 2n \quad \text{--- (2)}$$

$$= 2^3 T\left(\frac{n}{2^3}\right) + 3n \quad \text{--- (3)}$$

$$\vdots$$
$$= 2^k T\left(\frac{n}{2^k}\right) + kn \quad \text{--- (4)}$$

Assume

$$\frac{n}{2^k} = 1$$

$$n = 2^k \Rightarrow k = \log_2 n$$

$$T(n) = n \cdot T\left(\frac{n}{n}\right) + \log_2 n \cdot n$$

$$T(n) = n \cdot T(1) + n \log_2 n$$

$$= n + n \log_2 n$$

$$= O(n \log_2 n)$$

Time complexity for worst case:-

$$T(n) = \begin{cases} 1 & \text{if } n=0 \\ T(n-1) + n & \text{if } n > 0 \end{cases}$$

BS
not here

$$T(n) = T(n-1) + n \quad \text{--- ①}$$

$$T(n) = T(n-2) + (n-1) + n \quad \text{--- ②}$$

$$T(n) = T(n-3) + (n-2) + (n-1) + n \quad \text{--- ③}$$

$$T(n) = T(n-4) + (n-3) + (n-2) + (n-1) + n \quad \text{--- ④}$$

let assume $n-k=0$

$$= T(n-k) + (n-(k-1)) + (n-(k-2)) + \dots + (n-1) + n$$

$$= T(n-k) + (n-k+1) + (n-k+2) + \dots + (n-1) + n$$

$$= T(n-k) + (n-k+1) + (n-k+2) + \dots + (n-1) + n$$

$$= T(0) + 1 + 2 + \dots + (n-1) + n$$

$$= T(0) + \frac{n(n+1)}{2}$$

$$= 1 + \frac{n(n+1)}{2}$$

$$= O(n^2)$$

Note:-

Select pivot as middle element

Select Random element as pivot

**

MERGE SORT:-

In merge sort based on the middle element the list can be divided into left sublist and Right sublist.

Again consider left sublist findout mid value again it can be divide into left sublist 1 & Right sublist 1, until we get 1 element in the list.

then combine ^(apply merge sort) all the elements

In the same way right sublist also divide the list until we get 1 element in the list.

Finally combined left & Right sort. list.

(eg:-)

20	50	30	10	40	60
----	----	----	----	----	----

Consider the list, Now find out middle element $m = \frac{l+h}{2}$

$$\frac{1+6}{2} = 3.5 = 3$$

The list 20, 50 & 30

20	50	30
1	2	3

find out mid value $m = \left\lfloor \frac{1+h}{2} \right\rfloor = \frac{1+3}{2} = \frac{4}{2} = 2$

Now the mid element is 2

Hence

20	50	30
1	2	3

Now again find out mid element $m = \left\lfloor \frac{1+h}{2} \right\rfloor = \frac{1+2}{2} = 1$

The final elements are

20	50	30
1	2	3

Now combine first two elements, hence the list is

20	50
----	----

Now combine index 1, 2 and 3

20	30	50
1	2	3

The list

10	40	60
4	5	6

find out middle value $m = \left\lfloor \frac{1+h}{2} \right\rfloor = \frac{4+6}{2} = \frac{10}{2} = 5$

Now the mid element is 5

Hence

10	40	60
4	5	6

Now again find out mid element $m = \left\lfloor \frac{1+h}{2} \right\rfloor = \left\lfloor \frac{4+5}{2} \right\rfloor = \frac{9}{2} = 4$

The final elements are

10	40	60
4	5	6

Now combine first two elements hence the list is

10	40
4	5

Now combine index 4, 5 and 6

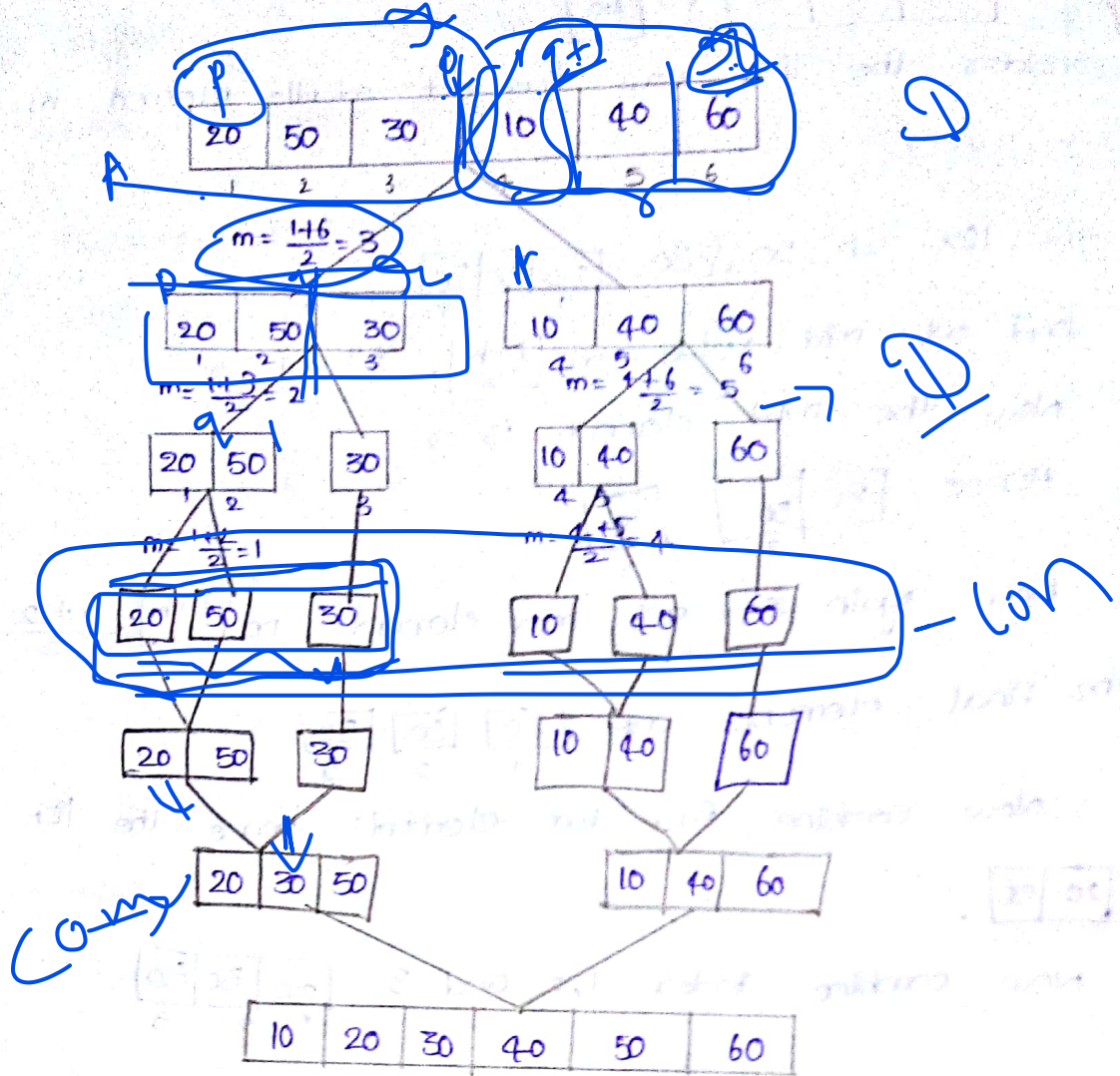
10	40	60
4	5	6

Now combine the indexes [1-3] and [4-6]

20	30	50	10	40	60
1	2	3	4	5	6

Hence the final sorted list is

10	20	30	40	50	60
----	----	----	----	----	----



eg: 65, 70, 60, 75, 85, 80, 55, 50, 45, 57

consider the list: [65, 70, 60, 75, 85, 80, 55, 50, 45, 57] (Indices 1 to 10)

Find out middle element $m = \frac{1+10}{2} = \frac{11}{2} = 5$

the list: [65, 70, 60, 75, 85] (Indices 1 to 5)

Find out mid value $m = \frac{1+5}{2} = \frac{6}{2} = 3$

Now the mid element is 3

Hence: [65, 70, 60] [75, 85] (Indices 1 to 5)

Now again find out mid element $m = \frac{1+3}{2} = 2$

[65, 70] [60] [75, 85] (Indices 1 to 5)

Now again find out mid element $m = \frac{1+2}{2} = 1$

[65] [70] [60] [75, 85] (Indices 1 to 5)

Now again find out mid value $m = \frac{1+5}{2} = \frac{6}{2} = 3$

The mid element is 4

65	70	60	75	85
1	2	3	4	5

Now combine first two elements hence the list is

65	70
1	2

Now combine indexes 1, 2 & 3

60	65	70
----	----	----

Now combine indexes 1, 2, 3 & 4

60	65	70	75
----	----	----	----

Now combine indexes 1, 2, 3, 4 & 5

60	65	70	75	80
1	2	3	4	5

The list

80	55	50	45	57
6	7	8	9	10

Find out middle value $m = \frac{1+10}{2} = \frac{11}{2} = 5.5$

Now the middle element is 8, hence

80	55	50
6	7	8

45	57
9	10

Now again find out mid element $m = \frac{6+8}{2} = \frac{14}{2} = 7$

Now the middle element is 7, hence

80	55	50
6	7	8

45	57
9	10

again find out middle element $m = \frac{6+7}{2} = \frac{13}{2} = 6.5$

Now the mid value is 6,

80	55	50	45	57
6	7	8	9	10

Again find out middle value $m = \frac{6+10}{2} = \frac{16}{2} = 8$

Now the mid value is 8,

80	55	50	45	57
6	7	8	9	10

Now combine first two elements hence the list is

80	55
6	7

Now combine indexes 6, 7 & 8

50	55	80
6	7	8

Now combine indexes 6, 7, 8 & 9

45	50	55	80
6	7	8	9

Now combine indexes 6, 7, 8, 9 & 10

45	50	55	57	80
6	7	8	9	10

Now combine indexes [1-5] & [6-10]

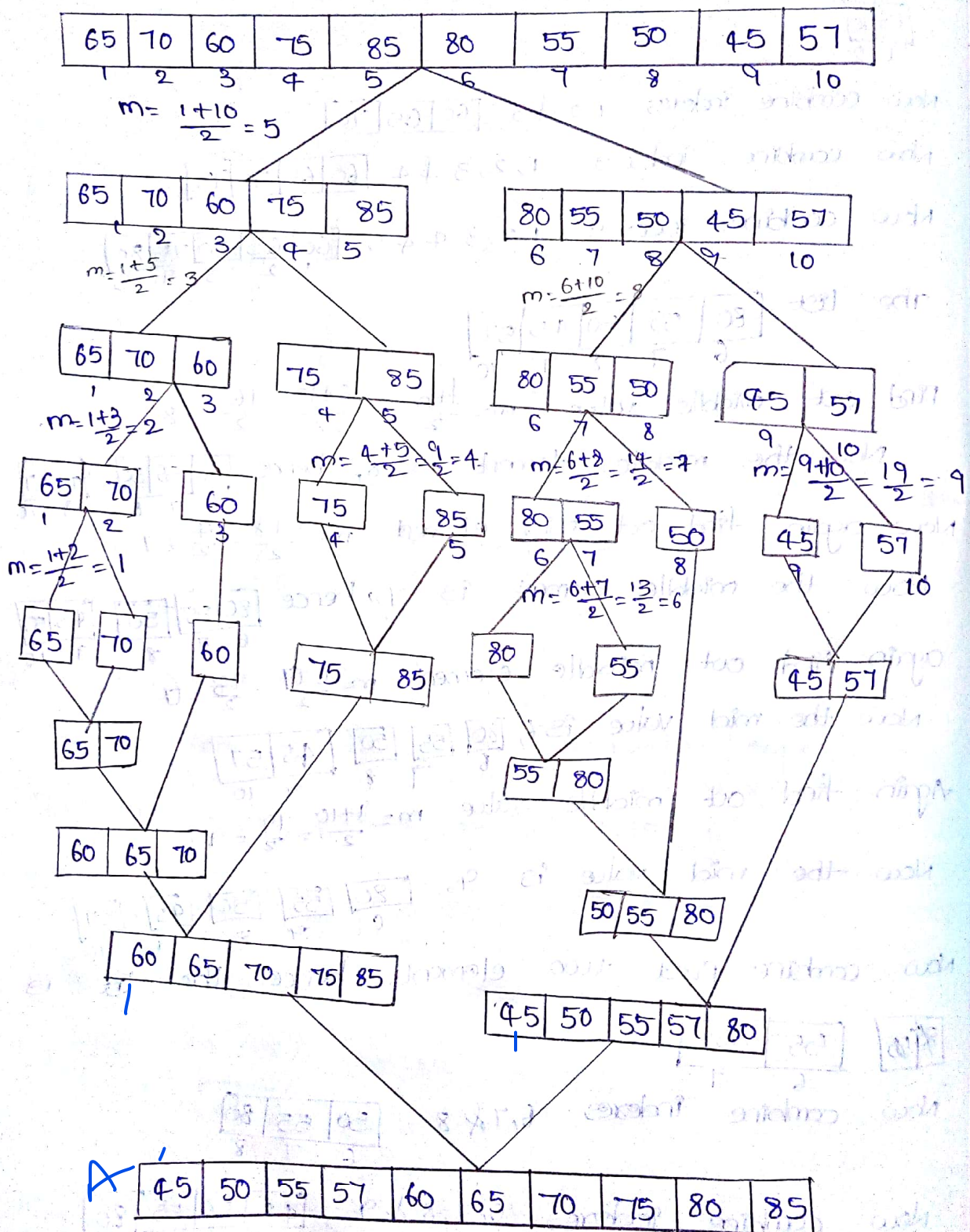
✓

60	65	70	75	85
1	2	3	4	5

45	50	55	57	80
6	7	8	9	10

Hence the final sorted list is

45	50	55	57	60	65	70	75	80	85
1	2	3	4	5	6	7	8	9	10



P

310	285	179	652	351	423	861	254	450	520
1	2	3	4	5	6	7	8	9	10

$$m = \frac{1+10}{2} = 5$$

310	285	179	652	351
1	2	3	4	5

$$m = \frac{1+5}{2} = 3$$

423	861	254	450	520
6	7	8	9	10

$$m = \frac{6+10}{2} = \frac{16}{2} = 8$$

310	285	179
1	2	3

$$m = \frac{1+3}{2} = 2$$

652	351
4	5

$$m = \frac{4+5}{2} = 4$$

423	861	254
6	7	8

$$m = \frac{6+8}{2} = \frac{14}{2} = 7$$

450	520
9	10

$$m = \frac{9+10}{2} = 9$$

310	285	179
1	2	3

$$m = \frac{1+2}{2} = 1$$

652	351
4	5

423	861	254
6	7	8

$$m = \frac{6+7}{2} = 6$$

450	520
9	10

310	285	179
1	2	3

351	652
4	5

423	861	254
6	7	8

450	520
9	10

285	310
1	2

179	285	310
1	2	3

351	652
4	5

423	861
6	7

450	520
9	10

179	285	310	351	652
1	2	3	4	5

254	423	450	520	861
6	7	8	9	10

179	254	285	310	351	423	450	520	652	861
1	2	3	4	5	6	7	8	9	10

Algorithm:-

Merge Sort algorithm:-

Algorithm MergeSort(low, high)

```
{
  if (low < high) // if there are more than one element
  {
    mid =  $\lfloor \frac{low+high}{2} \rfloor$ ; // Divide p into sub problems
    // - find where to split the set.
    → MergeSort(midlow, high); // solve the sub problems
    → MergeSort(low, mid, high); // combine the solutions
    } Merge(low, mid, high);
  }
}
```

Merge:-

Merging two sorted sub arrays using auxiliary storage

Algorithm Merge(low, mid, high)

P q r

```
{
  h := low;
  i := low;
  j := mid + 1;
  while (h <= mid) & (j <= high)
  {
    if (a[h] <= a[j]) then
    {
      b[i] = a[h];
      h := h + 1;
    }
    else
    {
      b[i] = a[j];
      j := j + 1;
    }
    i := i + 1;
  }
}
```

for $k := j$ to high do

{

$b[i] := a[k];$

$i := i + 1;$

}

else

{

for $k := j$ to mid do

{

$b[i] := a[k];$

$i = i + 1;$

}

for $k := low$ to high do

$a[k] := b[k];$

}

Time Complexity:-

Best

Note:- In merge sort the time complexity is, $O(n \log n)$

→ Best case $O(n \log n)$

Average case $O(n \log n)$

Worst case $O(n \log n)$

Recurrence Relation:-

$$T(n) = \begin{cases} 1 & \text{if } n=1 \\ 2T\left(\frac{n}{2}\right) + n & \text{if } n>1 \end{cases}$$

① $T(n) = 2T\left(\frac{n}{2}\right) + n \rightarrow$

$$T(n) = 2 \left[2T\left(\frac{n}{2^2}\right) + \frac{n}{2} \right] + n$$

$$= 2^2 + \left(\frac{n}{2^2}\right) + n + n$$

$$= 2^2 + \left(\frac{n}{2^3}\right) + 2n \rightarrow$$

$$= 2^3 + \left(\frac{n}{2^3}\right) + 3n \rightarrow$$

$\vdots$

$$= 2^k + \left(\frac{n}{2^k}\right) + kn \rightarrow$$

$$\begin{aligned} \therefore T(n) &= 2T\left(\frac{n}{2}\right) + n \\ T\left(\frac{n}{2}\right) &= 2T\left(\frac{n}{2^2}\right) + \frac{n}{2} \\ T\left(\frac{n}{2^2}\right) &= 2^2 T\left(\frac{n}{2^3}\right) + \frac{n}{2^2} \end{aligned}$$

Assume $\frac{n}{2} = 1 \Rightarrow n = 2^k$
 $k = \log_2 n$

$$T(n) = n \cdot T\left(\frac{n}{2}\right) + \log_2 n \cdot n$$

$$= n \cdot T(1) + n \log_2 n$$

$$= n + n \log_2 n$$

$$= O(n \log_2 n)$$

Strassen's Matrix Multiplication:-

* In general matrix multiplication is the no. of columns in the 1st matrix = the no. of rows in the 2nd matrix.

* Let us consider $A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$ $B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$

* The resultant matrix $C = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}$ where

$$c_{11} = a_{11} * b_{11} + a_{12} * b_{21}$$

$$c_{12} = a_{11} * b_{12} + a_{12} * b_{22}$$

$$c_{21} = a_{21} * b_{11} + a_{22} * b_{21}$$

$$c_{22} = a_{21} * b_{12} + a_{22} * b_{22}$$

* Here we have to compute 8 multiplications hence the time complexity of matrix multiplication is $O(n^3)$

* In case of 4×4 matrix we are going to compute large no. of multiplications needed.

* Let us assume

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix} \text{ and}$$

$$B = \begin{pmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{42} & b_{43} & b_{44} \end{pmatrix}$$

* Now the above matrix can be divided into 4 equal 2×2 matrices

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix} \begin{matrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{matrix}$$

$$B = \begin{pmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{42} & b_{43} & b_{44} \end{pmatrix} \begin{matrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{matrix}$$

* Now finally $C = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}$

* The recurrence relation for general matrix multiplication is

$$T(n) = \begin{cases} 1 & \text{if } n \leq 2 \\ 8T(n/2) + n^2 & \text{if } n > 2 \end{cases}$$

* Now the strassen's reduce the time complexity is

$$T(n) = \begin{cases} 1 & \text{if } n \leq 2 \\ 7T(n/2) + n^2 & \text{if } n > 2 \end{cases}$$

* Instead of using 8 multiplications strassen's matrix multiplications use only 7 multiplications.

* Hence the time complexity is $O(n^{2.81})$

* Strassen's introduce 7 different formula's are

$$\begin{aligned} P &= (A_{11} + A_{22}) \cdot (B_{11} + B_{22}) \\ Q &= B_{11} (A_{21} + A_{22}) \\ R &= A_{11} (B_{12} - B_{22}) \\ S &= A_{22} (B_{21} - B_{11}) \\ T &= B_{22} (A_{11} + A_{12}) \\ U &= (A_{21} - A_{11}) \cdot (B_{11} + B_{12}) \\ V &= (B_{21} + B_{22}) \cdot (A_{12} - A_{22}) \end{aligned}$$

$\therefore \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$
 $\xrightarrow{T(+)}$
 $\begin{matrix} \uparrow 11 & 12 \\ S(-) & \begin{pmatrix} 21 & 22 \end{pmatrix} \\ \downarrow Q(+), R(-) \end{matrix}$
 $\begin{matrix} B_{11} & A_{11} & A_{12} & B_{22} \\ 11 & 11 & 12 & 22 \end{matrix}$

$$C_{11} = P + S - T + V$$

$$C_{12} = R + T$$

$$C_{21} = Q + S$$

$$C_{22} = P + R - Q + U$$

ROT
badd

Eg:-

$$A = \begin{pmatrix} 1 & 3 \\ 7 & 5 \end{pmatrix} \quad B = \begin{pmatrix} 6 & 7 \\ 3 & 8 \end{pmatrix} \quad \text{find } C$$

$$\text{Here consider } A_{11} = 1, A_{12} = 3, A_{21} = 7, A_{22} = 5$$

$$B_{11} = 6, B_{12} = 7, B_{21} = 3, B_{22} = 8$$

$$P = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$$

$$= (1 + 5) \cdot (6 + 8) = (6) \cdot (14) = 84$$

$$Q = B_{11} \cdot (A_{21} + A_{22})$$

$$= 6(7 + 5)$$

$$= 6(12)$$

$$= 72$$

$$R = A_{11} \cdot (B_{12} - B_{22})$$

$$= 1(7 - 8) = -1$$

$$S = A_{22} \cdot (B_{21} - B_{11})$$

$$= 5(3 - 6) = 5(-3) = -15$$

$$T = B_{22} \cdot (A_{11} + A_{12})$$

$$= 8(1 + 3) = 8(4) = 32$$

$$U = (A_{21} - A_{11}) \cdot (B_{11} + B_{12})$$

$$= (7 - 1)(6 + 7) = (6)(13) = 78$$

$$V = (B_{21} + B_{22}) \cdot (A_{12} - A_{22})$$

$$= (3 + 8)(3 - 5) = (11)(-2) = -22$$

$$C_{11} = P + S - T + V$$

$$= 84 + (-15) - 32 + (-22)$$

$$= 84 - 15 - 32 - 22$$

$$= 15$$

$$C_{12} = R + T = -1 + 32 = 31$$

$$C_{21} = Q + S = 72 + (-15) = 57$$

$$C_{22} = P + R - Q + U$$

$$= 84 + (-1) - 72 + 78$$

$$= 89$$

hence the resultant matrix $C = \begin{pmatrix} 15 & 31 \\ 57 & 89 \end{pmatrix}$

eg:- $A = \begin{pmatrix} 4 & 5 \\ 6 & 9 \end{pmatrix}$ $B = \begin{pmatrix} 12 & 20 \\ 3 & 14 \end{pmatrix}$ Then find $A \times B = C$.

Here consider $A_{11} = 4$ $A_{12} = 5$ $A_{21} = 6$ $A_{22} = 9$

$$B_{11} = 12$$
 $B_{12} = 20$ $B_{21} = 3$ $B_{22} = 14$

$$P = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$$

$$= (4 + 9)(12 + 14) = (13)(26) = 338$$

$$Q = B_{11} \cdot (A_{21} + A_{22}) = 12(6 + 9) = 12(15) = 180$$

$$R = A_{11} \cdot (B_{12} - B_{22}) = 4(20 - 14) = 4(6) = 24$$

$$S = A_{22}(B_{21} - B_{11}) = 9(3 - 12) = 9(-9) = -81$$

$$T = B_{22}(A_{11} + A_{12}) = 14(4 + 5) = 14(9) = 126$$

$$U = (A_{21} - A_{11})(B_{11} + B_{12}) = (6 - 4)(12 + 20) = (2)(32) = 64$$

$$V = (B_{21} + B_{22})(A_{12} - A_{22}) = (3 + 14)(5 - 9) = (17)(-4) = -68$$

$$C_{11} = P + S - T + V$$

$$338 - 275 = 63$$

$$= 338 + (-81) - 126 - 68 = 338 - 275 = 63$$

$$C_{12} = R + T = 24 + 126 = 150$$

$$C_{21} = Q + S = 180 + (-81) = 99$$

$$C_{22} = P + R - Q + U = 338 + 24 - 180 + 64 = 394 - 180 = 214$$

Hence the resultant matrix $C = \begin{pmatrix} 63 & 150 \\ 99 & 214 \end{pmatrix}$

Algorithm

Algorithm MatrixMul (A, B, n)

{

if ($n \leq 2$)

{

C = 4 formula's

}

else

{

mid ... $n/2$

MM (A_{11} , B_{11} , $n/2$) + MM (A_{12} , B_{21} , $n/2$);

MM (A_{11} , B_{12} , $n/2$) + MM (A_{12} , B_{22} , $n/2$);

MM (A_{21} , B_{11} , $n/2$) + MM (A_{22} , B_{21} , $n/2$);

MM (A_{21} , B_{12} , $n/2$) + MM (A_{22} , B_{22} , $n/2$);

}

Time complexity:

$O(n^{2.81})$

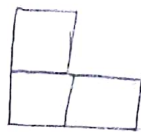
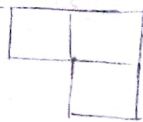
24/12/18.

Prepared by Suman

* Defective chess board Algorithm problem:-

→ Given chessboard of size $n \times n$ i.e. $n = 2^k$.

A chessboard contains exactly one defective indicated by square, (or) add to fill chessboard with help of L-shaped tiles. These L-shaped tiles are called Triaminoes.



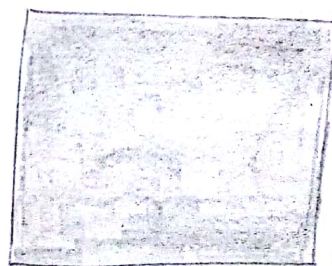
→ In a chessboard total no. of squares = $n \times n$, where $n = 2^k \Rightarrow$ i.e. $2^k \times 2^k \Rightarrow 2^{2k}$.

→ In a chessboard, no. of non-defective squares = $2^{2k} - 1$

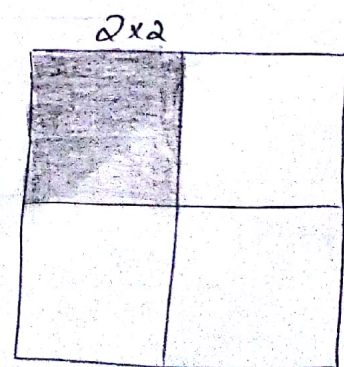
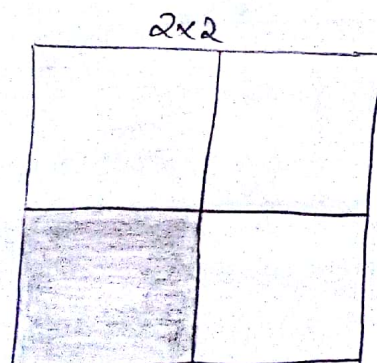
→ This number is always divisible by 3.

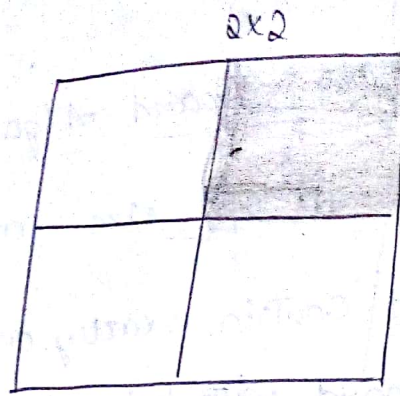
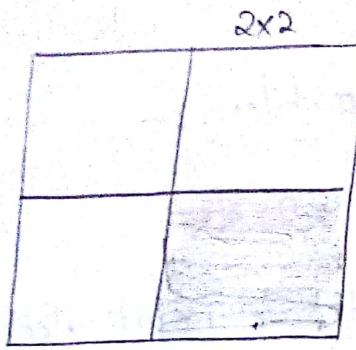
→ In 1×1 chessboard only one defective square.

→ In 1×1 chessboard the no. of Non-defective chessboard is 0.



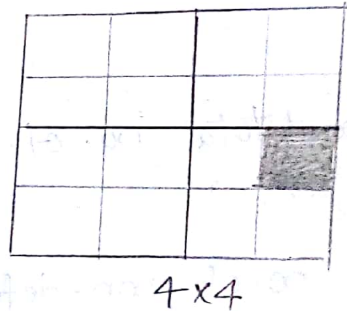
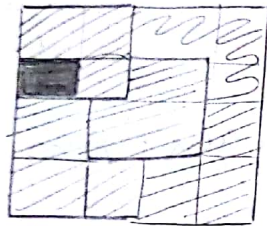
→ In 2×2 chessboard, exactly one defective square & remaining three are comes under to Non-defective squares.





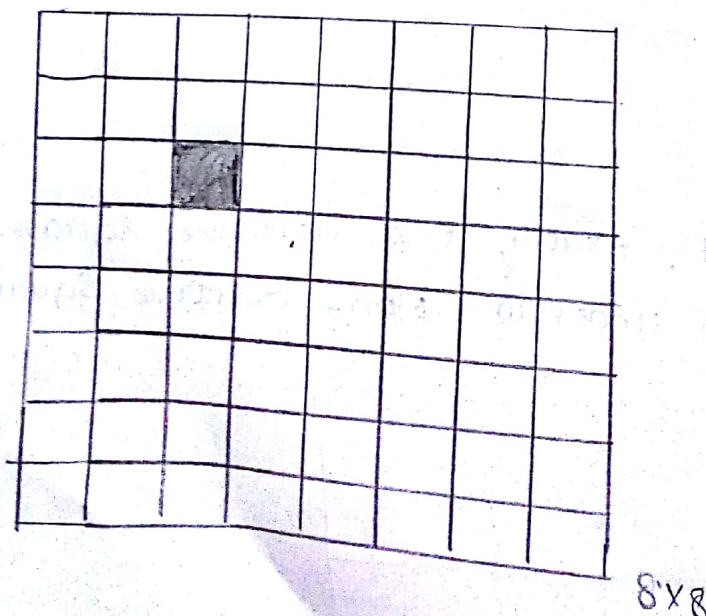
→ In 4x4 chessboard, exactly one defective square.

→ The no. of non-defective squares are 15.



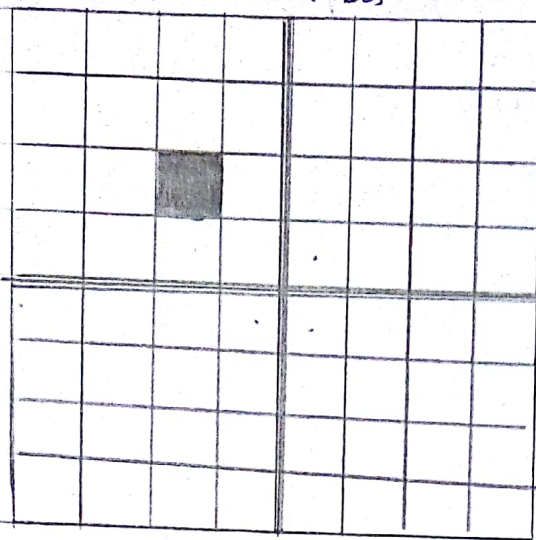
→ size $K=1$, i.e. 2x2 problem has a simple solution, if the problem is very large then it can be divided into smaller ones finally combine solutions to smaller problems.

Ex:- Tiling 8x8 Defective chessboard.



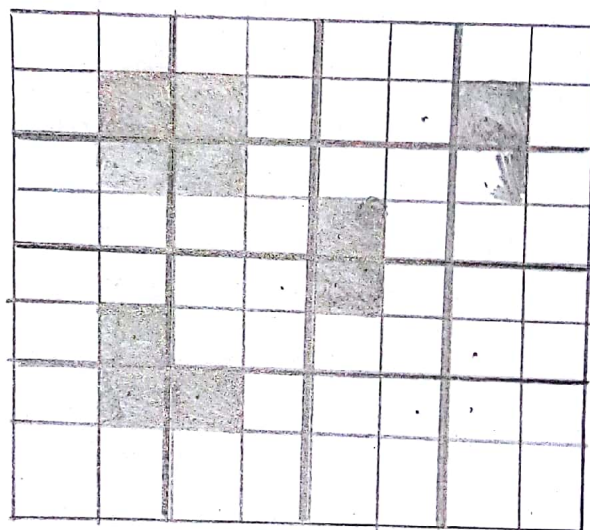
→ Now, the 8×8 chessboard can be divided into 4×4 chessboard.

→ Now, introduce, artificial triaminos.



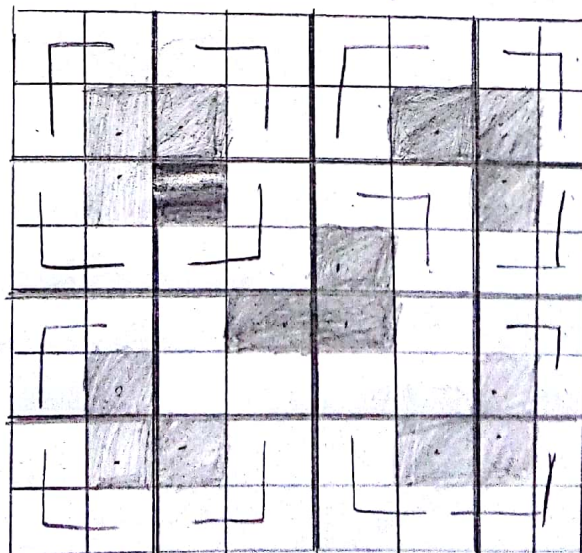
4×4

→ Now, again ^{each} 4×4 can be divided into possible no. of 2×2 chessboard.



→ Now add triaminos for each & every 2×2 chessboard.

→ Now Combine all the solutions



Note :- The Time Complexity of the defective chessboard is $O(n^2)$ i.e., $\Theta(n^2)$